

Angular Moderno

34 - Refino Profissional e Organizacao Final

2026

SENAI - CRTI

34 — Refino Profissional e Organização Final

Nesta aula vamos finalizar o projeto e transformá-lo em um produto acadêmico de qualidade.

Qualidade do Código

Melhorar a estrutura e legibilidade

Boas Práticas

Aplicar organização profissional

UX

Ajustar experiência do usuário

Arquitetura

Revisar estrutura do projeto

Entrega Final

Preparar o projeto para entrega

Contexto da Aula

Até aqui, construímos um sistema completo com todas as funcionalidades essenciais. Agora vamos evoluir o projeto com foco em **qualidade e organização profissional**.



Catálogo de Produtos

Listagem e visualização de produtos da loja



Carrinho de Compras

Adição, remoção e controle de itens



Checkout

Finalização de pedidos com formulário



Login e Acesso

Controle de autenticação e autorização



Área Administrativa

Painel protegido para gestão do sistema

Etapa 1 — Parar o Servidor

Antes de iniciar as alterações, é necessário parar o servidor Angular para evitar conflitos durante as modificações.

 Sempre pare o servidor antes de realizar múltiplas alterações em arquivos do projeto.

No terminal, pressione:

CTRL + C

Justificativa: Evita conflitos durante múltiplas alterações em arquivos e garante que tudo será recompilado corretamente ao final.

Por que parar o servidor?

- Evita recompilações parciais
- Previne erros de cache
- Garante rebuild limpo
- Melhora estabilidade do ambiente

Etapa 2 – Organização de Modelos

Vamos criar um **model** para padronizar o uso de produtos no projeto, centralizando a estrutura de dados.

Arquivo a ser criado

```
src/app/core/models/produto-loja.ts
```

Objetivo

Evitar repetição de tipos dentro dos componentes e services, centralizando a estrutura de dados em um único lugar.

Benefícios da centralização

- Sem repetição de tipos
Definição única reutilizada em todo o projeto
- Padronização
Estrutura de dados consistente em toda a aplicação
- Manutenção facilitada
Alterações em um único arquivo refletem em todo o projeto

Código completo - produto-loja.ts

app/core/models/produto-loja.ts

```
// Modelo criado para evitar tipos repetidos em diferentes arquivos.  
// Este tipo representa o produto já adaptado para uso interno na loja.  
export type ProdutoLoja = {  
  nome: string;  
  preco: number;  
};
```

Etapa 3 — Refatoração do ProdutosService

Arquivo: `src/app/core/services/produtos.service.ts`

1

Importar o novo model

```
import { ProdutoLoja } from '../models/produto-loja';
```

2

Ajustar retorno do método

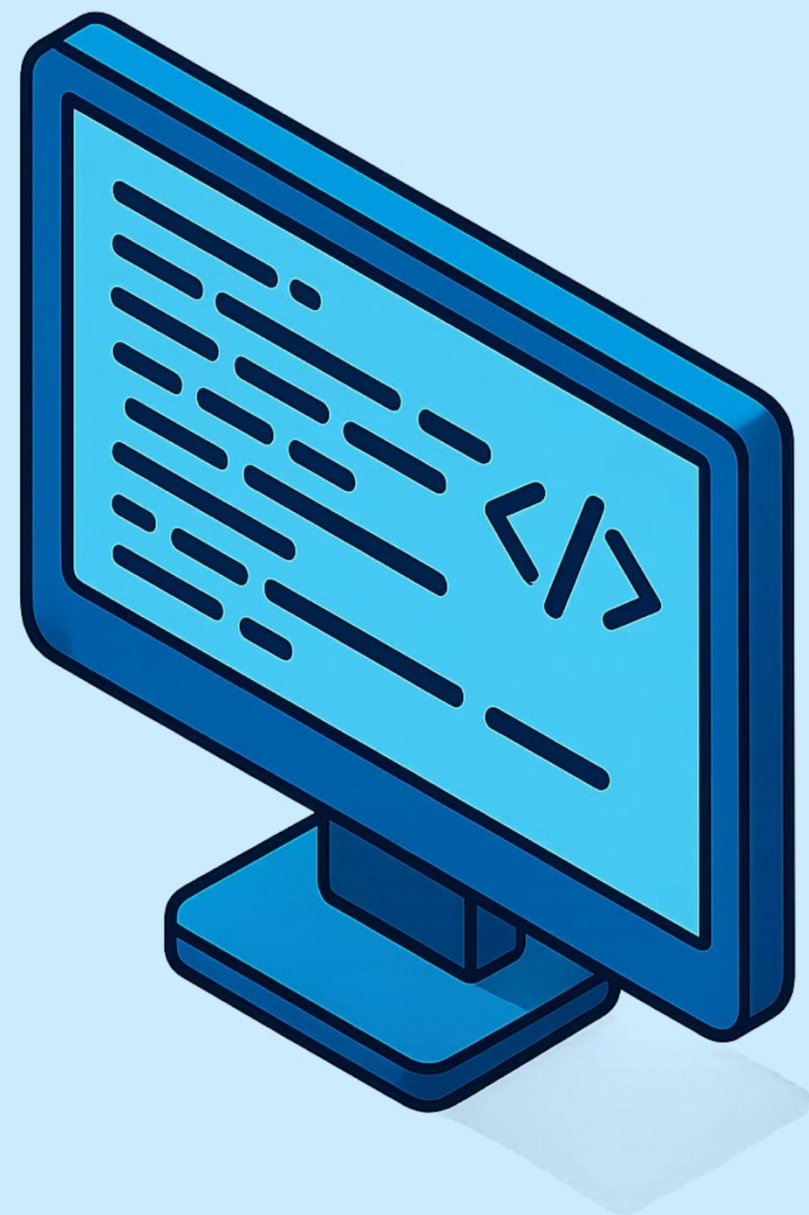
Ajustar o retorno do método `transformarProdutos` para `ProdutoLoja[]`

3

Responsabilidade clara

- Buscar dados da API
- Transformar dados para uso interno

📄 **Justificativa:** Separação de responsabilidades e padronização de dados em toda a aplicação.



Código completo - produtos.service.ts

app/core/services/produtos.service.ts

```
import { HttpClient } from '@angular/common/http';
import { Injectable, inject } from '@angular/core';

import { ProdutoLoja } from '../models/produto-loja';

type ProdutoApi = {
  title: string;
  price: number;
};

@Injectable({
  providedIn: 'root',
})
export class ProdutosService {
  // Imports organizados e tipo ProdutoLoja movido para model próprio.
  private http = inject(HttpClient);

  private readonly API = 'https://fakestoreapi.com/products';
  // private readonly API = 'https://fakestoreapi.com/products-erro';

  buscarProdutos() {
    return this.http.get<ProdutoApi[]>(this.API);
  }

  transformarProdutos(dados: ProdutoApi[]): ProdutoLoja[] {
    return dados.map((produto) => ({
      nome: produto.title,
      preco: produto.price,
    }));
  }
}
```

Etapa 4 — Limpeza do Componente ListaProdutos

Arquivo: `src/app/features/produtos/lista-produtos/lista-produtos.ts`

O que remover

`adicionarProduto()`

Método de teste — **remover**

`substituirProdutos()`

Método de teste — **remover**

Será necessário importar/usar `ProdutoLoja` e `RouterLink`.

Signals a manter organizados

- `produtos`
- `carregando`
- `erro`
- `produtoSelecioneado`

✔ **Justificativa:** Remover código de teste melhora legibilidade e qualidade final do projeto.

Etapa 5 — Aplicação de `toFixed`

Arquivo: `src/app/features/produtos/lista-produtos/lista-produtos.ts`

Alteração — Criar valor formatado

```
valorTotalFormatado = computed(() =>  
  this.valorTotal().toFixed(2)  
);
```

 **Justificativa:** Padronizar exibição de valores monetários com duas casas decimais em toda a aplicação.

Por que usar `toFixed(2)`?

- Consistência visual
- Todos os valores exibem sempre duas casas decimais
- Padrão monetário
- Formato correto para valores em reais (R\$)
- Evita arredondamentos
- Controle preciso sobre a exibição numérica

Código completo - `lista-produtos.ts`

```
import { Component, computed, effect, inject, signal } from '@angular/core';
import { RouterLink } from '@angular/router';
import { MatButtonModule } from '@angular/material/button';

import { CarrinhoFacade } from '../../core/facades/carrinho.facade';
import { ItemCarrinho } from '../../core/models/item-carrinho';
import { ProdutoLoja } from '../../core/models/produto-loja';
import { ProdutosService } from '../../core/services/produtos.service';
import { Produto } from '../produto/produto';

@Component({
  selector: 'app-lista-produtos',
  imports: [Produto, MatButtonModule, RouterLink],
  templateUrl: './lista-produtos.html',
  styleUrls: ['./lista-produtos.css'],
})
export class ListaProdutos {
  // Componente limpo para representar apenas a tela de catálogo.
  private produtosService = inject(ProdutosService);
  carrinhoFacade = inject(CarrinhoFacade);

  produtos = signal<ProdutoLoja[]>([]);
  produtoSelecionado = signal<string | null>(null);
  carregando = signal(true);
  erro = signal<string | null>(null);

  totalProdutos = computed(() => this.produtos().length);

  valorTotal = computed(() => this.produtos().reduce((total, item) => total + item.preco, 0));

  valorTotalFormatado = computed(() => this.valorTotal().toFixed(2));

  constructor() {
    this.carregarProdutos();

    // Mantém apenas um efeito útil para a experiência do usuário.
    effect(() => {
      if (typeof document !== 'undefined') {
        document.title = `(${this.totalProdutos()}) Minha Loja`;
      }
    });
  }
}
```

`app/features/produtos/lista-produtos/lista-produtos.ts`

```
carregarProdutos() {
  this.erro.set(null);
  this.carregando.set(true);

  this.produtosService.buscarProdutos().subscribe({
    next: (dados) => {
      const produtos = this.produtosService.transformarProdutos(dados);
      this.produtos.set(produtos);
      this.carregando.set(false);
    },
    error: (erro) => {
      console.error('Erro ao carregar produtos:', erro);
      this.erro.set('Erro ao carregar produtos. Verifique sua conexão e tente novamente.');
```

Etapa 6 — Ajustes no Template `lista-produtos.html`

Arquivo: `src/app/features/produtos/lista-produtos/lista-produtos.html`

1 Remover botões de teste existentes

Limpar o template de elementos que não fazem parte da aplicação final

2 Melhorar organização visual com seções

Estruturar o HTML com seções bem definidas para melhor leitura

✔ **Justificativa:** Melhor UX e exibição correta de valores monetários.

Etapa 7 — Uso de `<hr>` para Separação Visual

Arquivo:

`src/app/features/produtos/lista-produtos/lista-produtos.html`

Inserir `<hr />` entre as seções principais do template para melhorar a organização visual.

Onde inserir

01

Entre resumo e lista

Separar o bloco de resumo do grid de produtos

02

Entre lista e produto selecionado

Separar o grid do detalhe do produto selecionado

Impacto na UX

- Melhora a organização visual da página
- Facilita o entendimento do usuário
- Cria hierarquia clara entre seções
- Solução simples e eficaz



Pequenos detalhes visuais fazem grande diferença na experiência do usuário.

Código completo - `lista-produtos.html`

`app/features/produtos/lista-produtos/lista-produtos.html`

```
<!-- Catálogo final limpo, sem botões de teste usados nas aulas anteriores. -->
<section class="pagina-produtos">
  <header class="cabecalho-produtos">
    <div>
      <h2>Produtos</h2>
      <p>Escolha os produtos e adicione-os ao carrinho.</p>
    </div>

    <button mat-stroked-button color="primary" routerLink="/carrinho">
      Ver carrinho ({{ carrinhoFacade.quantidade() }})
    </button>
  </header>

  @if (carregando()) {
    <p>Carregando produtos...</p>
  }

  @if (erro()) {
    <section class="mensagem-erro">
      <p>{{ erro() }}</p>
      <button mat-raised-button color="primary" type="button" (click)="carregarProdutos()">
        Tentar novamente
      </button>
    </section>
  }

  @if (!carregando() && !erro()) {
    <section class="resumo-produtos">
      <p>Total de produtos: {{ totalProdutos() }}</p>
      <p>Valor total do catálogo: R$ {{ valorTotalFormatado() }}</p>
    </section>

    <hr />
  }
</section>
```

```
<section class="grid-produtos">
  @for (item of produtos(); track $index) {
    <app-produto
      [nome]="item.nome"
      [preco]="item.preco"
      (produtoSelecioneado)="exibirProduto($event)"
      (produtoAdicionado)="adicionarAoCarrinho($event)"
    />
  }
</section>

<hr />

<section class="produto-selecionado">
  <h3>Produto selecionado</h3>

  @if (produtoSelecioneado()) {
    <p class="texto-sucesso">{{ produtoSelecioneado() }}</p>
  } @else {
    <p class="texto-neutro">Nenhum produto selecionado.</p>
  }
</section>
</section>
```



Etapa 8 — Melhorias de CSS do Catálogo

Arquivo: `src/app/features/produtos/lista-produtos/lista-produtos.css`



pagina-produtos

Classe principal da página



cabecalho-produtos

Estilo do cabeçalho



grid-produtos

Layout em grade dos produtos



resumo-produtos

Bloco de resumo do catálogo

Melhorias de espaçamento

Ajustar margens, paddings e gaps para um layout mais respirado e profissional.

Responsividade

Aplicar `media queries` para adaptar o layout a diferentes tamanhos de tela.



Justificativa: Transformar layout simples em interface mais profissional e responsiva.

Código completo - `lista-produtos.css`

`app/features/produtos/lista-produtos/lista-produtos.css`

```
.pagina-produtos {
  display: flex;
  flex-direction: column;
  gap: 16px;
}

.cabecalho-produtos {
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.cabecalho-produtos h2 {
  margin-bottom: 4px;
}

.resumo-produtos {
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 12px;
  background-color: #fafafa;
}

.grid-produtos {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
}

.mensagem-erro {
  border: 1px solid #f3b7b7;
  border-radius: 8px;
  padding: 12px;
  background-color: #fff5f5;
}
```

```
.produto-selecionado {
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 12px;
}

.texto-sucesso {
  color: green;
  font-weight: bold;
}

.texto-neutro {
  color: gray;
}

@media (min-width: 700px) {
  .cabecalho-produtos {
    flex-direction: row;
    justify-content: space-between;
    align-items: center;
  }

  .grid-produtos {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (min-width: 1000px) {
  .grid-produtos {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

Etapa 9 — Criação do CSS do Checkout

Arquivo a ser criado, **caso não exista**: `src/app/features/checkout/checkout/checkout.css`

1

Layout do formulário

Melhorar a organização e espaçamento dos campos do formulário de checkout

2

Mensagens de sucesso

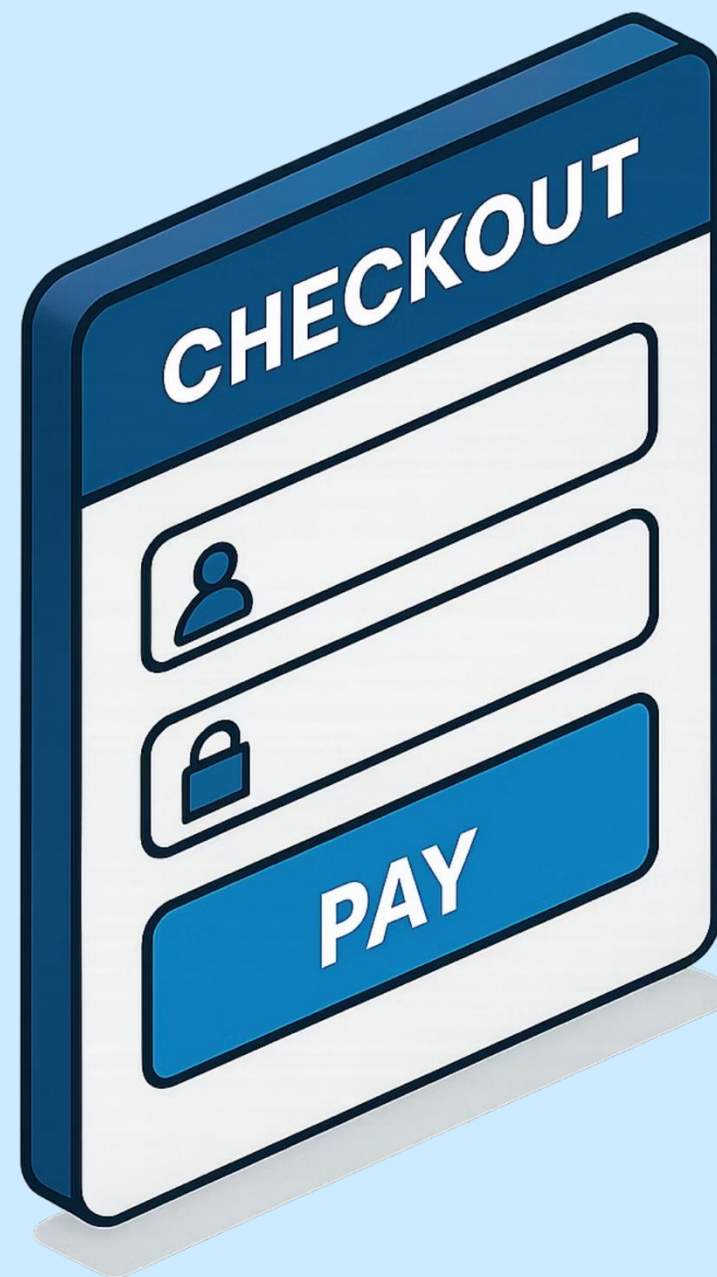
Destacar visualmente a confirmação do pedido com cores e estilos adequados

3

Alertas e avisos

Estilizar mensagens de erro e alerta para feedback claro ao usuário

✔ **Justificativa:** Interface final precisa transmitir clareza e feedback visual ao usuário.



Código completo do arquivo `checkout.css`

`app/features/checkout/checkout/checkout.css`

```
.pagina-checkout {
  max-width: 700px;
}

.bloco-checkout {
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 16px;
  margin: 16px 0;
}

.sucesso {
  border-color: #b7e4c7;
  background-color: #f1fff5;
}

.alerta {
  border-color: #f3b7b7;
  background-color: #fff5f5;
}
```

```
form {
  max-width: 500px;
}

input {
  width: 100%;
  padding: 8px;
  box-sizing: border-box;
}

small {
  display: block;
  color: red;
  margin-bottom: 8px;
}

button {
  margin-right: 8px;
  margin-top: 8px;
}
```

Etapa 10 — Ajustes no Carrinho

Arquivo: `src/app/features/carrinho/carrinho/carrinho.css`

Alterações a realizar

01

Organização dos itens

Melhorar a disposição visual de cada item do carrinho

02

Ajuste de espaçamentos

Margens e paddings para melhor leitura

03

Classes CSS

Revisar e refinar classes semânticas e reutilizáveis

Classes revisadas/refinadas

- `lista-carrinho` — container da lista
- `item-carrinho` — cada produto no carrinho
- `resumo-carrinho` — bloco de totais



Justificativa: Melhor leitura das informações do carrinho pelo usuário.

Código completo - **carrinho.css**

app/features/carrinho/carrinho/carrinho.css

```
.pagina-carrinho {
  max-width: 800px;
}

.lista-carrinho {
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.item-carrinho {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 12px;
}
```

```
.item-carrinho h3 {
  margin: 0 0 4px;
}

.item-carrinho p {
  margin: 0;
}

.resumo-carrinho {
  margin: 16px 0;
}

.acoes-carrinho {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}
```

Revisão da Arquitetura

Organização final do projeto adotada neste curso.

core

services Lógica de negócio e API	models Interfaces e tipos
facades Abstração de estado	guards Proteção de rotas
interceptors Middleware HTTP	

features

produtos Catálogo e listagem	carrinho Gestão do carrinho
checkout Finalização de pedidos	admin Painel administrativo

📄 **Justificativa:** Separação clara de responsabilidades e escalabilidade do projeto.

Etapa 11 — Retomar o Servidor

Comando no terminal

```
ng serve
```

Acessar no navegador

```
http://localhost:4200
```

Funcionalidades a testar



Produtos

Verificar listagem



Carrinho

Adicionar, remover e visualizar itens



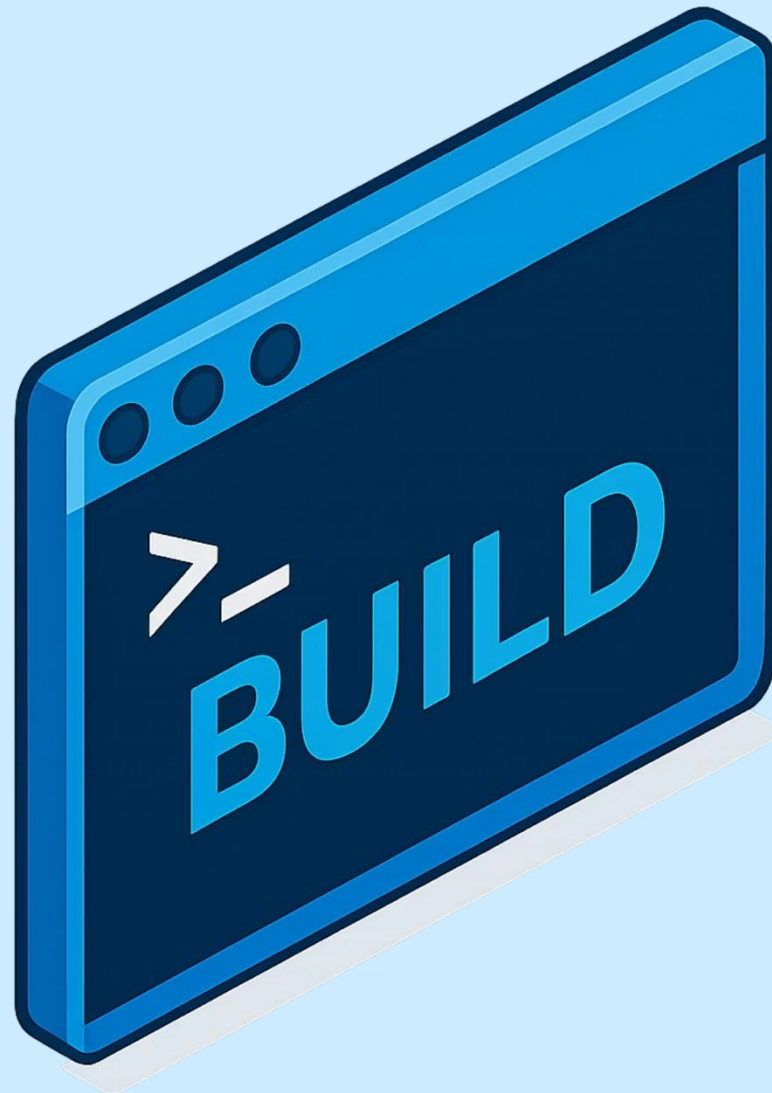
Checkout

Preencher formulário e finalizar pedido



Login e Admin

Autenticação e acesso à área protegida



Etapa 12 – Build Final do Projeto

Com todas as alterações realizadas e testadas, é hora de gerar o build de produção.

Executar no terminal

```
ng build
```

Resultado esperado

✔ Build concluído **sem erros!**
Projeto pronto para entrega!

✔ Mensagem exibida:
`Application bundle
generation complete.`

✔ Pasta criada:
`dist\meu-primeiro-app`

📄 **Justificativa:** Garantir que o projeto está completamente pronto para entrega acadêmica.

O que o build verifica?

Erros de TypeScript

Tipagem correta em todos os arquivos

Imports inválidos

Todos os módulos referenciados existem

Templates HTML

Bindings e diretivas corretos

Otimização

Minificação e tree-shaking automáticos

Conclusão da Aula

Parabéns! Este projeto agora representa um **produto acadêmico completo e profissional**.



Código Refinado

Removidos métodos de teste, imports organizados e tipos padronizados



UX Melhorada

Valores formatados, separadores visuais e CSS profissional



Arquitetura Organizada

Separação clara entre core e features com responsabilidades definidas



Pronto para Entrega

Build sem erros e projeto completo para avaliação acadêmica



Dúvidas

?

